Fundamentos de Inteligencia Artificial

Búsqueda heurística avanzada y MultiAgentes

José A. Montenegro

5 de febrero de 2026

Contenido

Búsqueda heurística avanzada
- IDA*
- Pathfinding

Búsqueda multiagente
- Algoritmo Minimax
- Poda Alfa-Beta
- Funciones de Evaluación

*IDA**

IDA^*

- Iterative-deepening A^* (IDA^*) dónde el límite de la búsqueda es definida utilizando la función de coste f(n) en vez de la profundidad.
- IDA^* nos da las ventajas de A^* sin el requisito de mantener todos los estados alcanzados en memoria,
- Por contramedida, vamos a visitar algunos estados varias veces.
- Es un algoritmo muy importante comúnmente utilizado para problemas que no caben en memoria.

IDA*

```
function IDA*(problem) returns a solution sequence
  inputs: problem, a problem
  static: f-limit, the current f- COST limit
          root, a node

  root ← MAKE-NODE(INITIAL-STATE[problem])
  f-limit ← f- COST(root)
  loop do
      solution, f-limit ← DFS-CONTOUR(root, f-limit)
      if solution is non-null then return solution
      if f-limit = ∞ then return failure; end

function DFS-CONTOUR(node, f-limit) returns a solution sequence and a new f- COST limit
  inputs: node, a node
          f-limit, the current f- COST limit
  static: next-f, the f- COST limit for the next contour, initially ∞

  if f- COST[node] > f-limit then return null, f- COST[node]
  if GOAL-TEST[problem](STATE[node]) then return node, f-limit
  for each node s in SUCCESSORS(node) do
      solution, new-f ← DFS-CONTOUR(s, f-limit)
      if solution is non-null then return solution, f-limit
      next-f ← MIN(next-f, new-f); end
  return null, next-f
```

IDA* en Python

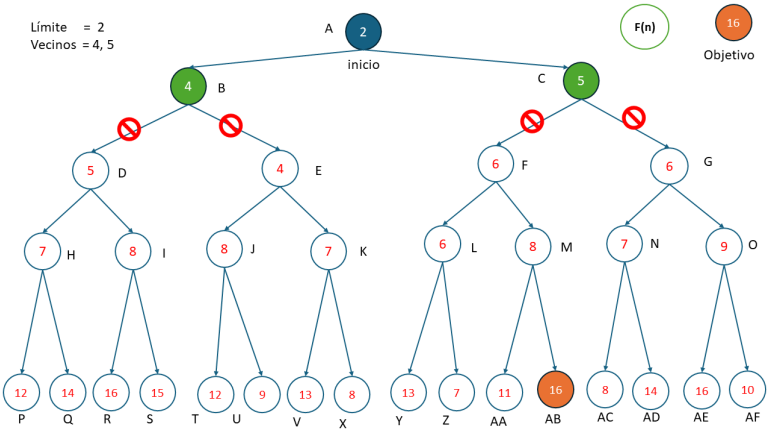
```
def ida_star(problem):
    bound = problem.f(Node(problem.initial_state))
    path = [Node(problem.initial_state)]
    while True:
        result, new_bound = depth_limited_search(problem, bound, path)
        if result == 'FOUND':
            return path
        if result == float('inf'):
            return 'No se encontró camino'
        bound = new_bound
        print("\n\n new bound", bound)
```

IDA* en Python

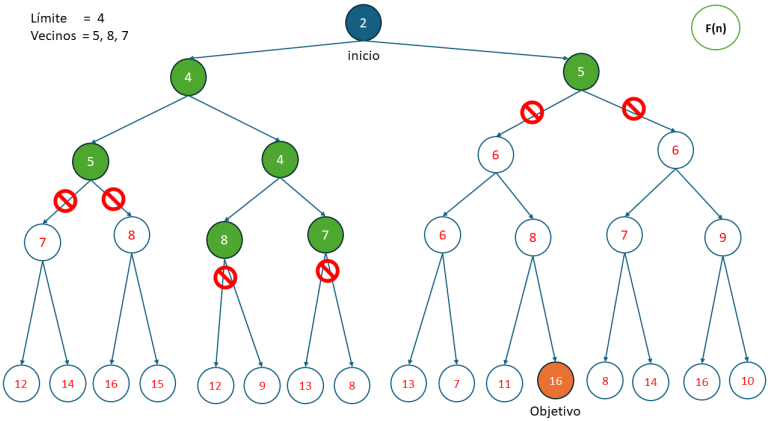
```python
def depth_limited_search(problem, bound, path,):

    node = path[-1]
    f = problem.f(node)
    print("NODE", node, "f",f)
    if problem.f(node) > bound:
        print("\t Poda NODE", node, "f",f)
        return f, 0
    if problem.is_goal(node.state):
        return 'FOUND', 0

    min_val = float('inf')
    for child_state, cost in problem.expand(node.state):
        if child_state not in [n.state for n in path]:
            child_node = Node(child_state, node, node.cost + cost)
            path.append(child_node)
            result, _ = depth_limited_search(problem, bound, path)
            if result == 'FOUND':
                return 'FOUND', 0
            if result < min_val:
                min_val = result
            path.pop()
    return min_val, min_val
```

*IDA**
Ejemplo Grafo

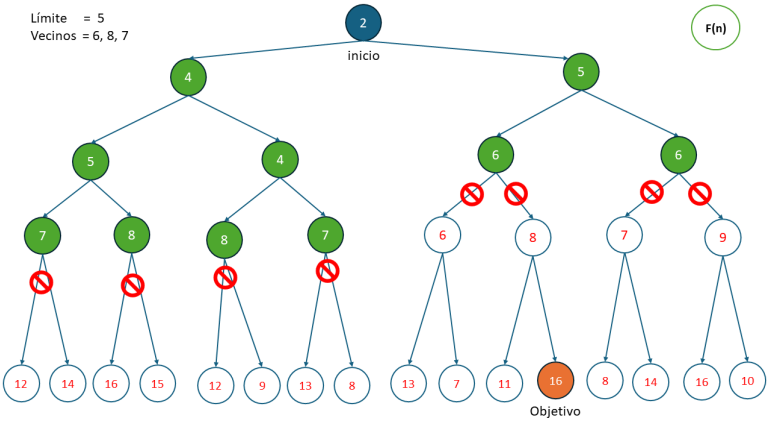
Ejemplo Grafo



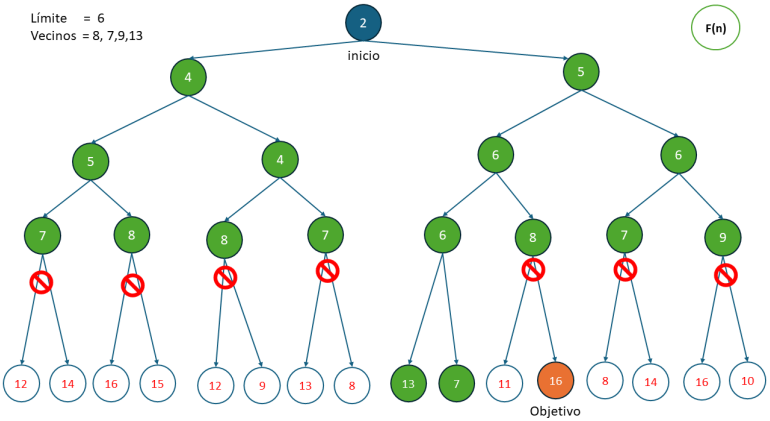
Ejemplo Grafo



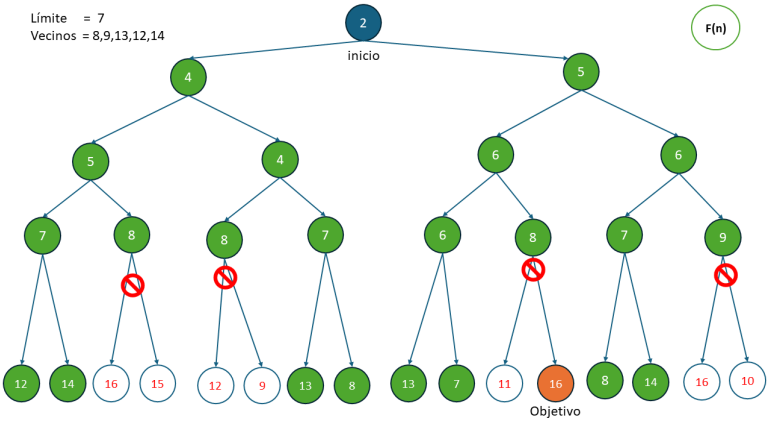
Ejemplo Grafo



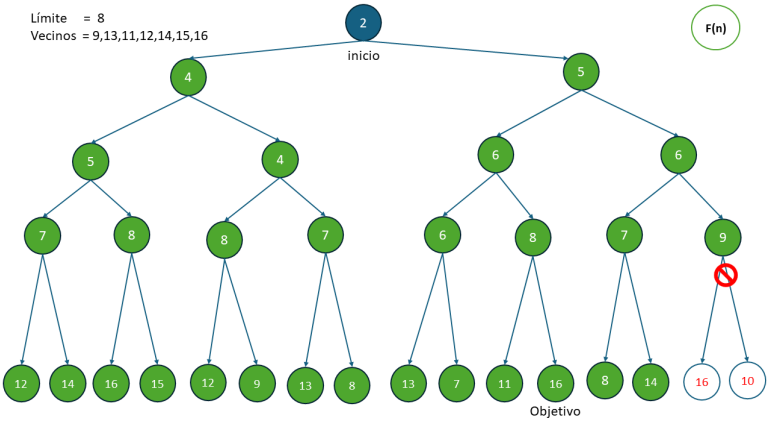
Ejemplo Grafo



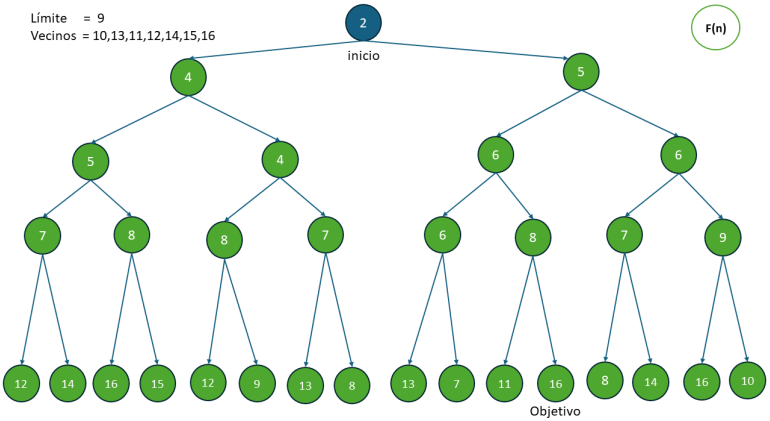
Ejemplo Grafo



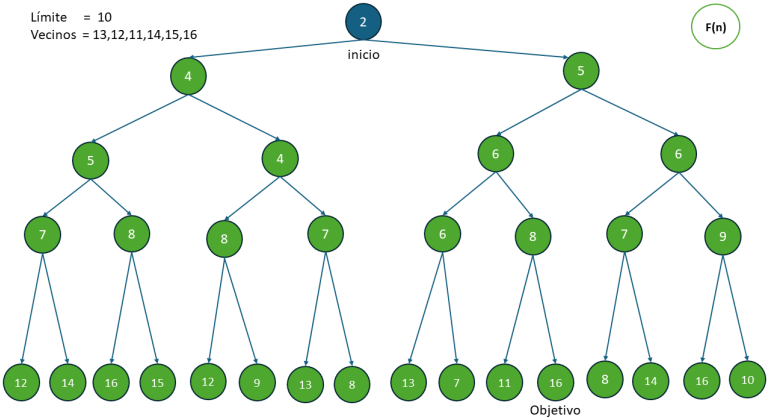
Ejemplo Grafo



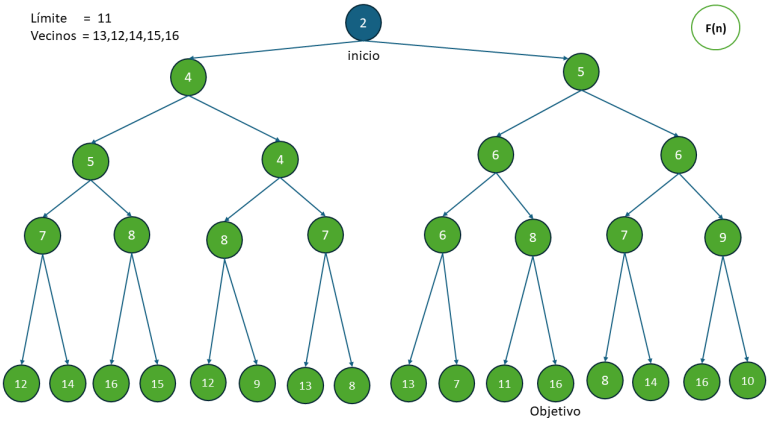
Ejemplo Grafo



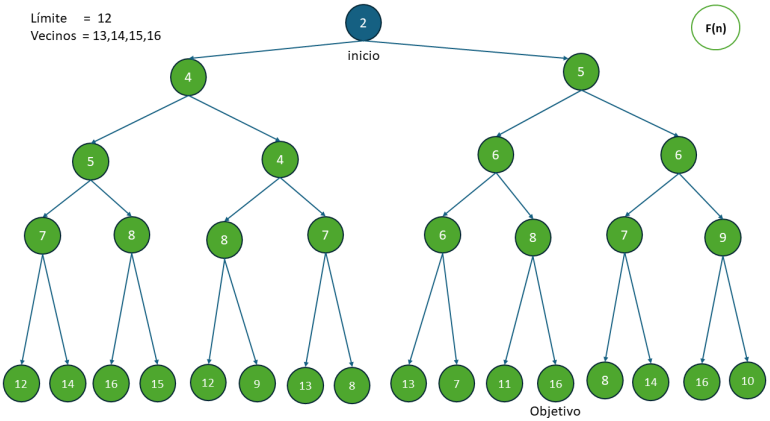
Ejemplo Grafo



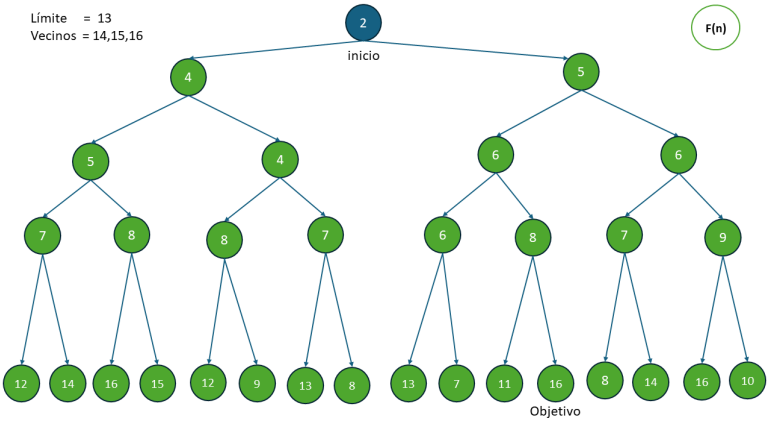
Ejemplo Grafo



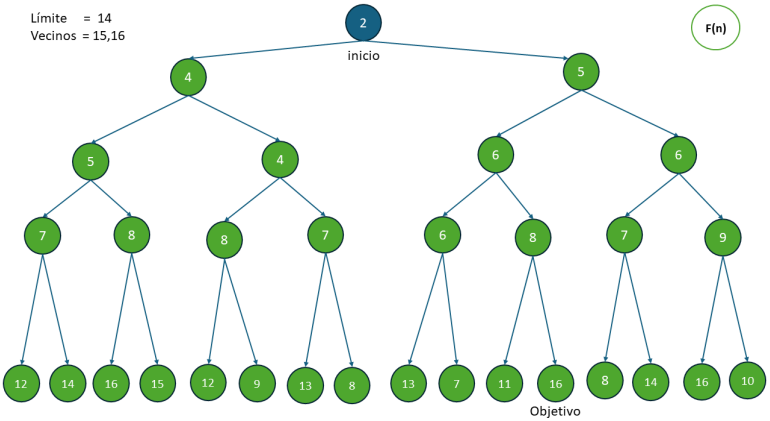
Ejemplo Grafo



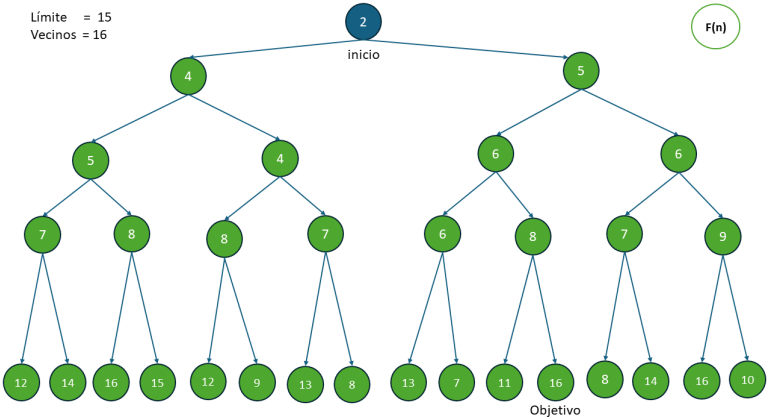
Ejemplo Grafo



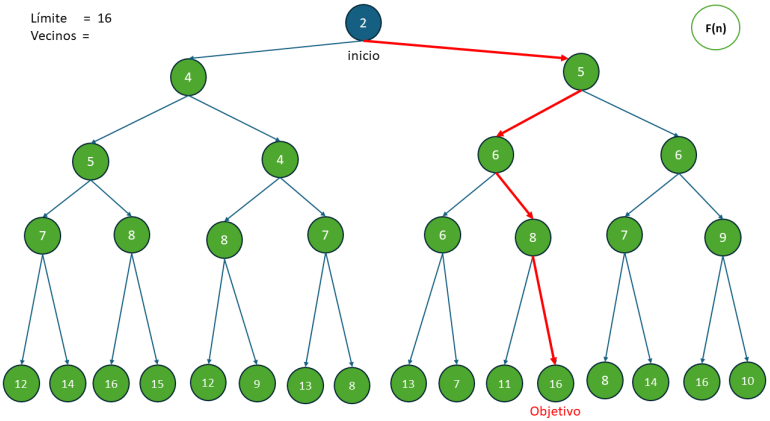
Ejemplo Grafo



Ejemplo Grafo



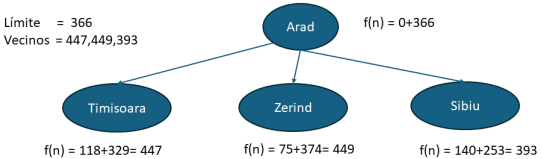
Ejemplo Grafo



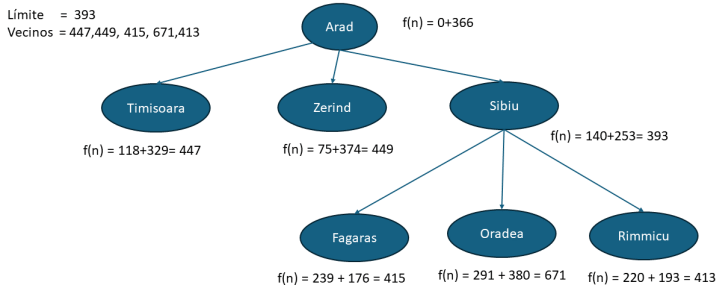
*IDA**

Ejemplo Rumania

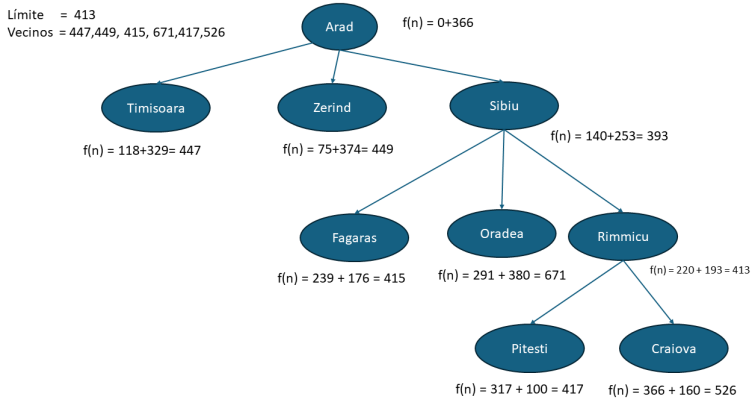
Ejemplo Rumania



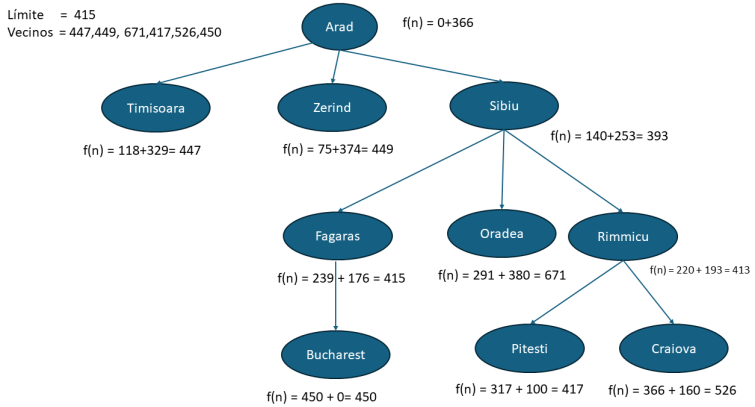
Ejemplo Rumania



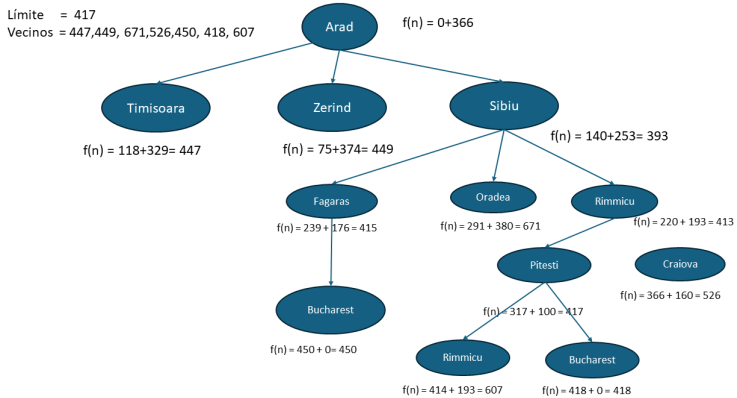
Ejemplo Rumania



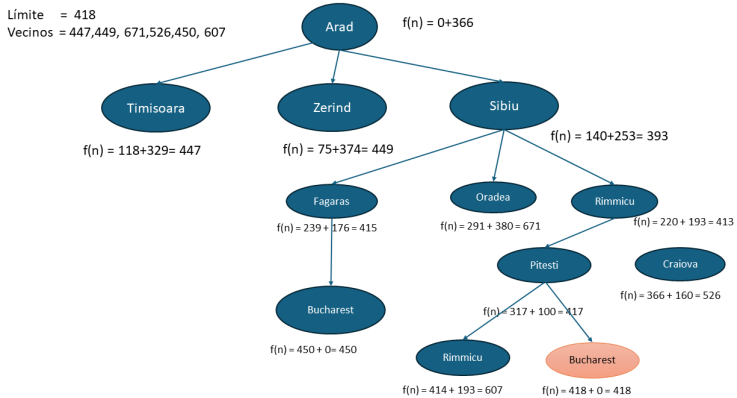
Ejemplo Rumania



Ejemplo Rumania



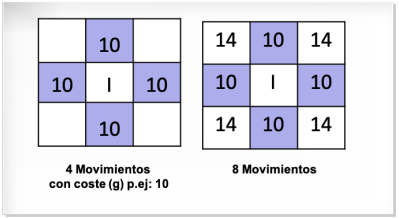
Ejemplo Rumania

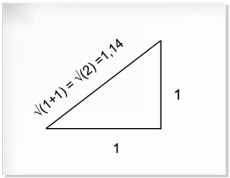


Path Finding

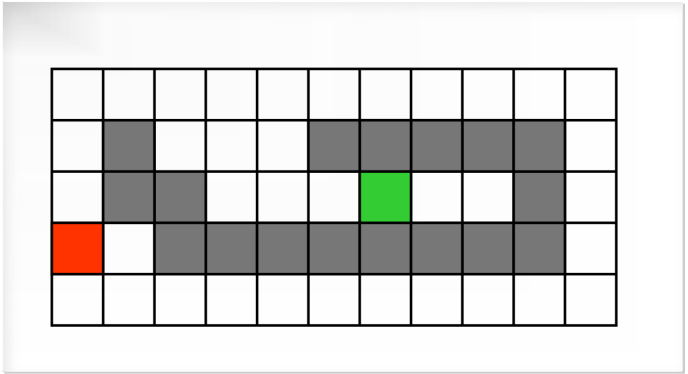
Path Finding

- Camino desde un nodo origen a un nodo destino, evitando obstáculos, sobre un mapa que está dividido en cuadrículas.
- Los movimientos permitidos en cada paso desde estado actual, dependen del problema:



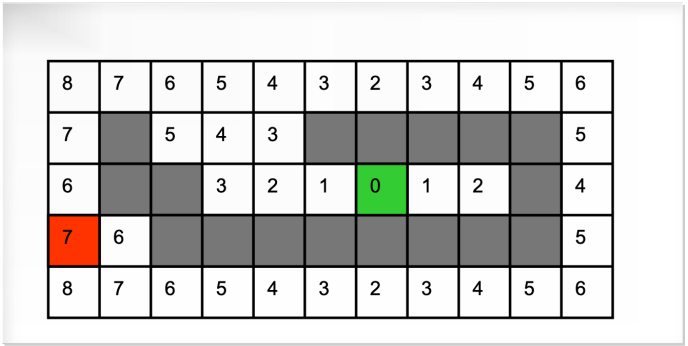


Ejemplo Path Finding 4 Movimientos



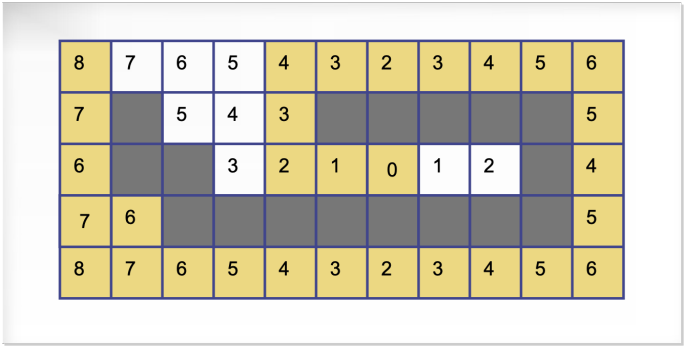
Ejemplo Path Finding 4 Movimientos

f(N) = h(N), con h(N) = Distancia Manhattan al objetivo



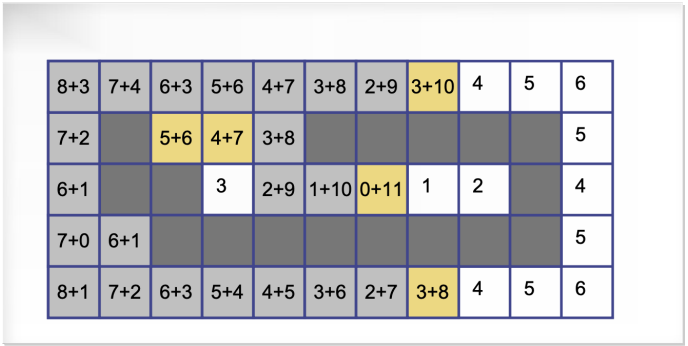
Ejemplo Path Finding 4 Movimientos

f(N) = h(N), con h(N) = Distancia Manhattan al objetivo

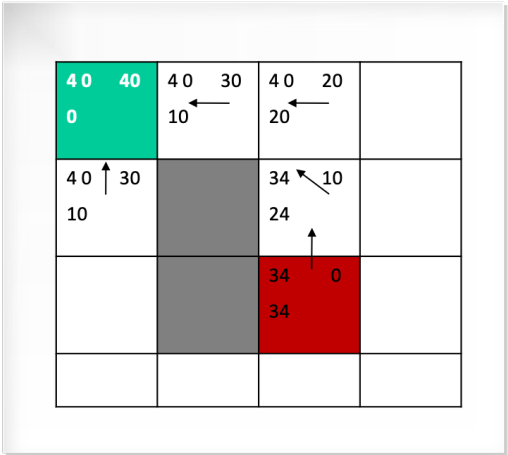


Ejemplo Path Finding 4 Movimientos

f(N) = h(h) + g(N), con h(N) = Distancia Manhattan al objetivo

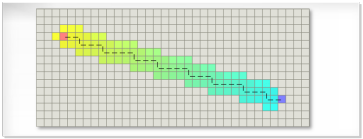


Ejemplo Path Finding 8 Movimientos

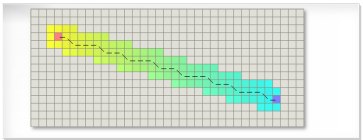


Path Finding Heurísticas

- Distancia Manhattan: h(n) = (abs(n.x-goal.x) + abs(n.y-goal.y)).



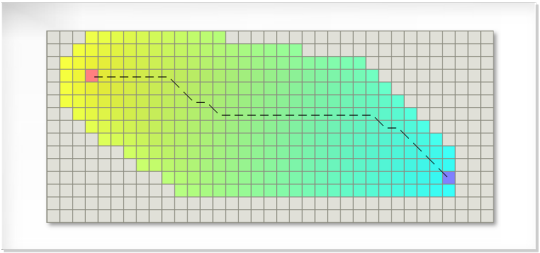
- Distancia Diagonal: h(n) = max(abs(n.x-goal.x), abs(n.y-goal.y))



Imágenes obtenidas de: http://theory.stanford.edu/~amitp/GameProgramming/Heuristics.html

Path Finding Heurísticas

- Distancia Euclídea: h(n) = $\sqrt{((n.x - goal.x)^2 + (n.y - goal.y)^2)}$.



Imágenes obtenidas de: http://theory.stanford.edu/~amitp/GameProgramming/Heuristics.html

PathFinding en Videojuegos

Abstracción Jerárquica (Hierarchical Abstraction)

- Reduce el espacio de búsqueda que tiene que ser explorado
- Realiza un cómputo del camino más rápido que A*
- Algunas implementaciones:
  - HPA* (Botea, Mueller, Schaeffer 2004)
  - PRA* (Sturtevant, Buro 2005)
  - HAA* (Harabor, Botea, 2008)

More info: `https://web.archive.org/web/20190411040123/http://aigamedev.com/open/article/clearance-based-pathfinding/`

Búsqueda multiagente

Búsqueda multiagente

- Entornos multiagente, donde cada agente debe considerar las acciones de los otros agentes.
- Juegos: Entornos competitivos donde los objetivos del agente están en conflicto, dan lugar a problemas de búsqueda entre adversarios.
  - Ajedrez, Otelo, Backgammon, Go
  - Ajedrez: Árbol de búsqueda tiene 10^{154} nodos.
- Capacidad de tomar decisión cuando no es factible calcular la decisión óptima.
  - **Poda**: Nos permiten ignorar partes del árbol búsqueda.
  - **Funciones evaluación**: Heurísticas que permiten aproximar la utilidad sin hacer búsqueda completa.

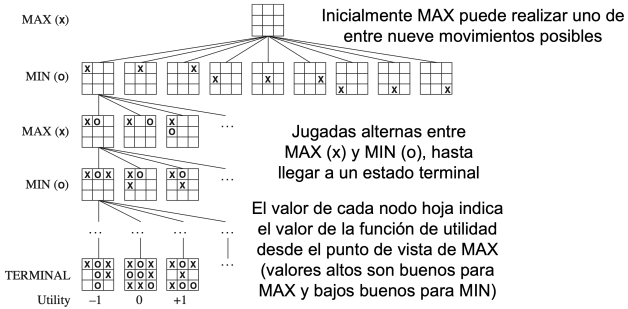
Elementos Búsqueda multiagente

- **Estado Inicial**: Posición tablero y jugador que mueve
- **Función sucesor**: Lista pares (movimiento, estado) , estado resultante.
- **Test terminal**: Cuando finaliza el juego. Estados terminales.
- **Función Utilidad**: Valor numérico a los estados terminales. Por ejemplo (suma nula): Triunfo +1, Pérdida -1, Empate 0
- **Árbol del juego**: Definido mediante estado inicial y los movimientos legales.

Búsqueda multiagente
Algoritmo Minimax

Algoritmo Minimax

- Juegos 2 jugadores (MAX y MIN). Primero mueve MAX y luego MIN por turnos hasta que termina
- Árbol del juego del Tres en Raya:



Algoritmo Minimax

1. Tiene por objetivo decidir un movimiento para MAX.
2. Hipótesis:
   - Jugador MAX trata de maximizar su beneficio (función de utilidad).
   - Jugador MIN trata de minimizar su pérdida.
   - Suponemos Jugadores juegan de forma óptima.
3. Aplicación algoritmo:
   - 3.1 Generar árbol entero hasta nodos terminales
   - 3.2 Aplicar función utilidad a nodos terminales
   - 3.3 Propagar hacia arriba para generar nuevos valores de utilidad para todos los nodos
     minimizando para MIN
     maximizando para MAX
   - 3.4 Elección jugada con máximo valor de utilidad

Algoritmo Minimax

```
function MINIMAX-DECISION(state) returns an action
  return arg max_{a ∈ ACTIONS(s)} MIN-VALUE(RESULT(state, a))

function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s, a)))
  return v

function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← ∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s, a)))
  return v
```

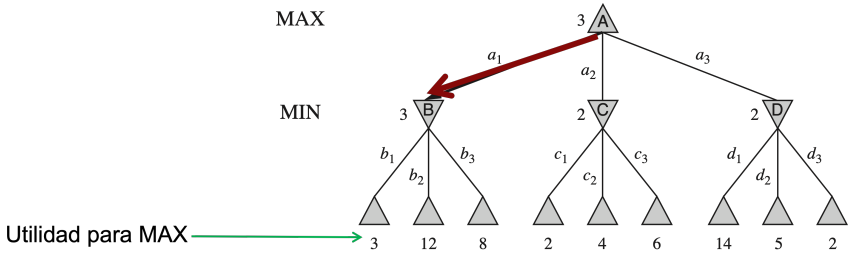
Algoritmo Minimax Ed. 4

```
function MINIMAX-SEARCH(game, state) returns an action
    player ← game.TO-MOVE(state)
    value, move ← MAX-VALUE(game, state)
    return move

function MAX-VALUE(game, state) returns a (utility, move) pair
    if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v ← −∞
    for each a in game.ACTIONS(state) do
        v2, a2 ← MIN-VALUE(game, game.RESULT(state, a))
        if v2 > v then
            v, move ← v2, a
    return v, move

function MIN-VALUE(game, state) returns a (utility, move) pair
    if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
    v ← +∞
    for each a in game.ACTIONS(state) do
        v2, a2 ← MAX-VALUE(game, game.RESULT(state, a))
        if v2 < v then
            v, move ← v2, a
    return v, move
```

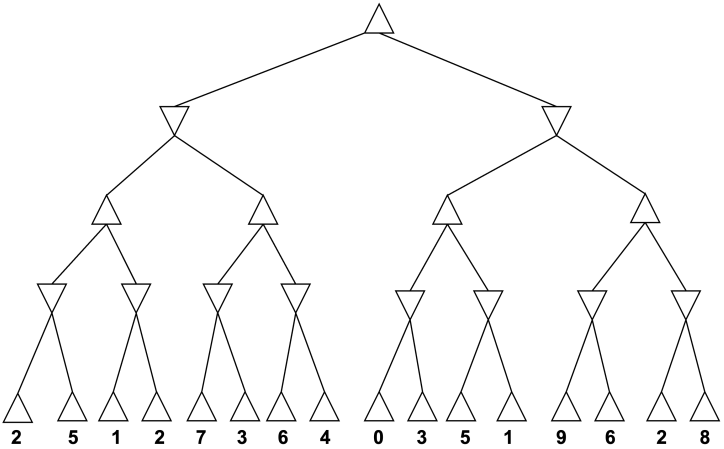
Algoritmo Minimax

MINIMAX(s)

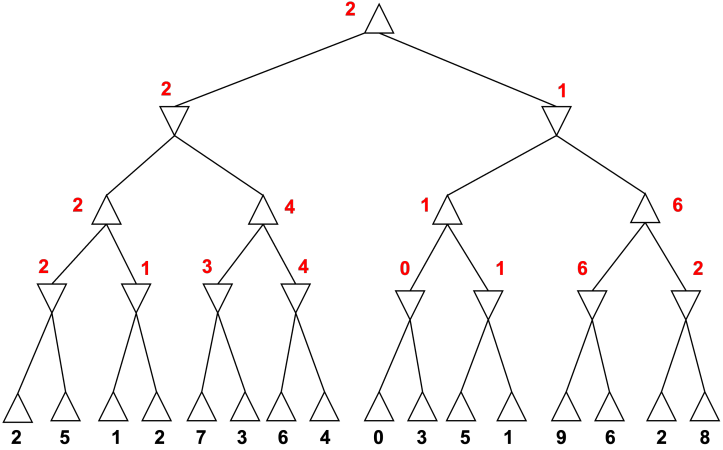
UTILIDAD(s)	if TERMINAL-TEST(s)
max_a MINIMAX (RESULT(s,a))	if PLAYES(s) = MAX
min_a MINIMAX (RESULT(s,a))	if PLAYES(s) = MIN



Ejemplo 1



Ejemplo 1



Búsqueda multiagente
Poda Alfa-Beta

Poda α-β

- No afecta al resultado final
- Su efectividad depende del orden en que se consideren los movimientos
- α valor de la mejor elección para MAX (valor más alto) que hemos encontrado en el camino hasta ahora
- β valor de la mejor elección para MIN (valor más bajo) que hemos encontrado en el camino hasta ahora

Poda α-β

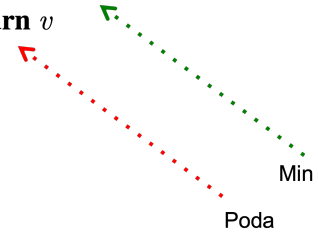
Alfa

Beta

function ALPHA-BETA-SEARCH($state$) **returns** an action
  $v \leftarrow$ MAX-VALUE($state, -\infty, +\infty$)
  return the $action$ in ACTIONS($state$) with value v

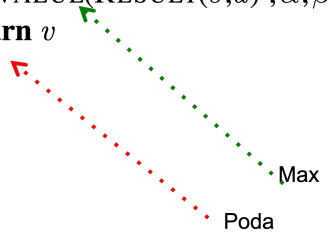
Poda α-β

function MAX-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  if TERMINAL-TEST($state$) **then return** UTILITY($state$)
  $v \leftarrow -\infty$
  for each a **in** ACTIONS($state$) **do**
    $v \leftarrow$ MAX(v, MIN-VALUE(RESULT(s,a), α, β))
    if $v \geq \beta$ **then return** v
    $\alpha \leftarrow$ MAX(α, v)
  return v



Poda α-β

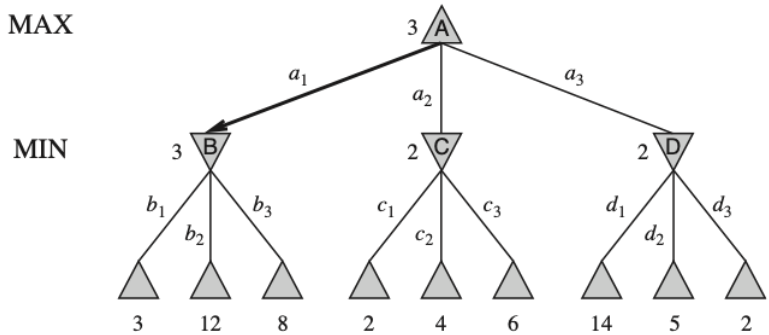
function MIN-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  if TERMINAL-TEST($state$) **then return** UTILITY($state$)
  $v \leftarrow +\infty$
  for each a **in** ACTIONS($state$) **do**
    $v \leftarrow$ MIN(v, MAX-VALUE(RESULT(s,a) ,α, β))
    if $v \leq \alpha$ **then return** v
    $\beta \leftarrow$ MIN(β, v)
  return v



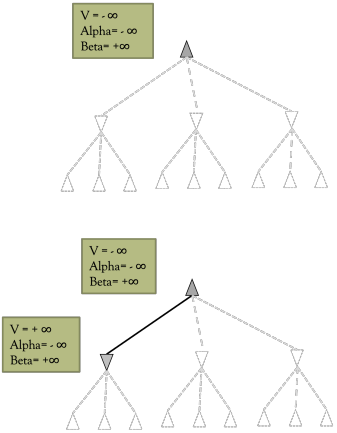
Poda α-β Ed. 4

```
function ALPHA-BETA-SEARCH(game, state) returns an action
  player ← game.TO-MOVE(state)
  value, move ← MAX-VALUE(game, state, −∞, +∞)
  return move

function MAX-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← −∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MIN-VALUE(game, game.RESULT(state, a), α, β)
    if v2 > v then
      v, move ← v2, a
      α ← MAX(α, v)
    if v ≥ β then return v, move
  return v, move

function MIN-VALUE(game, state, α, β) returns a (utility, move) pair
  if game.IS-TERMINAL(state) then return game.UTILITY(state, player), null
  v ← +∞
  for each a in game.ACTIONS(state) do
    v2, a2 ← MAX-VALUE(game, game.RESULT(state, a), α, β)
    if v2 < v then
      v, move ← v2, a
      β ← MIN(β, v)
    if v ≤ α then return v, move
  return v, move
```

Ejemplo Poda α-β



Ejemplo Poda α-β

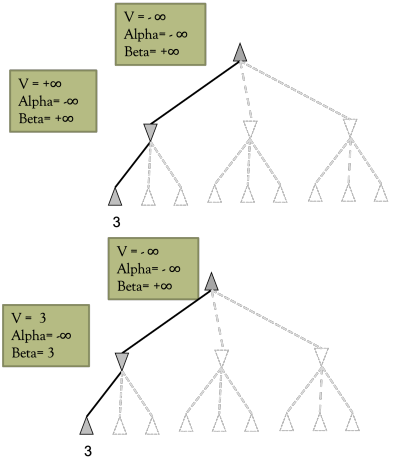


function ALPHA-BETA-SEARCH(*state*) **returns** an action
  $v \leftarrow$ MAX-VALUE($state, -\infty, +\infty$)
  return the *action* in ACTIONS(*state*) with value v

function MAX-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
  $v \leftarrow -\infty$
  for each a **in** ACTIONS(*state*) **do**
    $v \leftarrow$ MAX(v, MIN-VALUE(RESULT(s,a), α, β))
    if $v \geq \beta$ **then return** v
    $\alpha \leftarrow$ MAX(α, v)
  return v

function MIN-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)
  $v \leftarrow +\infty$
  for each a **in** ACTIONS(*state*) **do**
    $v \leftarrow$ MIN(v, MAX-VALUE(RESULT(s,a) , α, β))
    if $v \leq \alpha$ **then return** v
    $\beta \leftarrow$ MIN(β, v)
  return v

Ejemplo Poda α-β



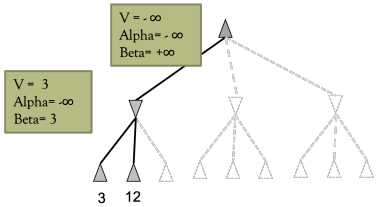
function MAX-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  if TERMINAL-TEST($state$) **then return** UTILITY($state$)
  $v \leftarrow -\infty$
  for each a **in** ACTIONS($state$) **do**
    $v \leftarrow$ MAX(v, MIN-VALUE(RESULT(s,a), α, β))
    if $v \geq \beta$ **then return** v
    $\alpha \leftarrow$ MAX(α, v)
  return v

Return 3

function MIN-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  if TERMINAL-TEST($state$) **then return** UTILITY($state$)
  $v \leftarrow +\infty$
  for each a **in** ACTIONS($state$) **do**
    $v \leftarrow$ MIN(v, MAX-VALUE(RESULT(s,a) , α, β))
    if $v \leq \alpha$ **then retur**
    $\beta \leftarrow$ MIN(β, v)
  return v

V= Min(+∞, 3)=3
3 ≤ −∞ FALSE
Beta= Min(+∞, 3)=3

Ejemplo Poda α-β



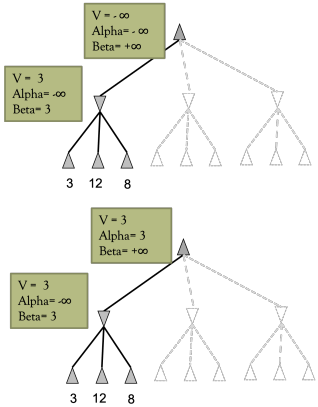
function MAX-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  if TERMINAL-TEST($state$) **then return** U
  $v \leftarrow -\infty$
  for each a **in** ACTIONS($state$) **do**
    $v \leftarrow$ MAX(v, MIN-VALUE(RESULT(s,a),α,β))
    if $v \geq \beta$ **then return** v
    $\alpha \leftarrow$ MAX(α, v)
  return v

Return 12

function MIN-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  if TERMINAL-TEST($state$) **then return** UTILITY($state$)
  $v \leftarrow +\infty$
  for each a **in** ACTIONS($state$) **do**
    $v \leftarrow$ MIN(v, MAX-VALUE(RESULT(s,a) ,α,β))
    if $v \leq \alpha$ **then return** v
    $\beta \leftarrow$ MIN(β, v)
  return v

V= Min(3, 12)=3
$3 \leq -\infty$ FALSE
Beta= Min(3, 12)=3

Ejemplo Poda α-β



function MAX-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  if TERMINAL-TEST($state$) **then return** U
  $v \leftarrow -\infty$
  for each a **in** ACTIONS($state$) **do**
    $v \leftarrow$ MAX(v, MIN-VALUE(RESULT(s,a), α, β))
    if $v \geq \beta$ **then return** v
    $\alpha \leftarrow$ MAX(α, v)
  return v

Return 8

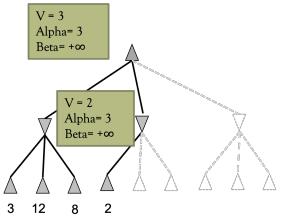
V= Max (- ∞,3,)=3
3 ≥ +∞ FALSE
Alfa= Max(- ∞, 3)=3

function MIN-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  if TERMINAL-TEST($state$) **then return** UTILITY($state$)
  $v \leftarrow +\infty$
  for each a **in** ACTIONS($state$) **do**
    $v \leftarrow$ MIN(v, MAX-VALUE(RESULT(s,a) , α, β))
    if $v \leq \alpha$ **then return** v
    $\beta \leftarrow$ MIN(β, v)
  return v

Return 3

V= Min(3, 8)=3
3 ≤ −∞ FALSE
Beta= Min(3, 8)=3

Ejemplo Poda α-β



function MAX-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  if TERMINAL-TEST($state$) **then return** U
  $v \leftarrow -\infty$
  for each a **in** ACTIONS($state$) **do**
    $v \leftarrow$ MAX(v, MIN-VALUE(RESULT(s, a), α, β))
    if $v \geq \beta$ **then return** v
    $\alpha \leftarrow$ MAX(α, v)
  return v

Return 2

V= Max (3,2)=3
3 ≥ +∞ FALSE
Alfa= Max(3,2)=3

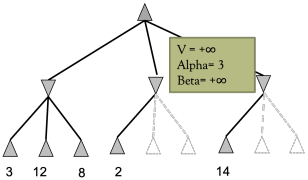
function MIN-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  if TERMINAL-TEST($state$) **then return** UTILITY($state$)
  $v \leftarrow +\infty$
  for each a **in** ACTIONS($state$) **do**
    $v \leftarrow$ MIN(v, MAX-VALUE(RESULT(s, a) , α, β))
    if $v \leq \alpha$ **then return** v
    $\beta \leftarrow$ MIN(β, v)
  return v

V= Min(+∞, 2)=2
2 ≤ 3 TRUE

Return 2

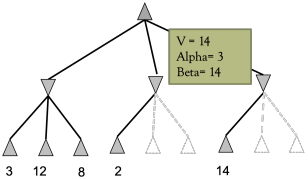
PODA!!!!!!

Ejemplo Poda α-β




```
function MAX-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s,a), α, β))
    if v ≥ β then return v
    α ← MAX(α, v)
  return v
```

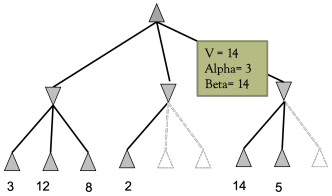
Return 14




```
function MIN-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← +∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s,a), α, β))
    if v ≤ α then return v
    β ← MIN(β, v)
  return v
```

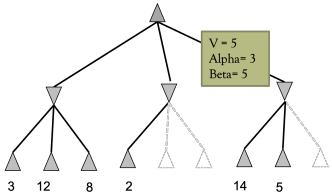
V= Min(+∞, 14)=14
14 ≤ 3 FALSE
Beta= Min(+∞, 14)=14

Ejemplo Poda α-β




```
function MAX-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s,a), α, β))
    if v ≥ β then return v
    α ← MAX(α, v)
  return v
```

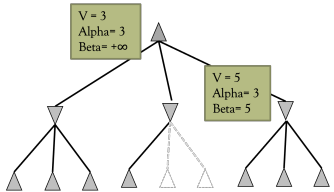
Return 5




```
function MIN-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← +∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s,a) , α, β))
    if v ≤ α then return v
    β ← MIN(β, v)
  return v
```

V= Min(14, 5)=5
5 ≤ 3 FALSE
Beta= Min(14, 5)=5

Ejemplo Poda α-β



function MAX-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  if TERMINAL-TEST($state$) **then return**
  $v \leftarrow -\infty$
  for each a **in** ACTIONS($state$) **do**
    $v \leftarrow$ MAX(v, MIN-VALUE(RESULT(s,a), α, β))
    if $v \geq \beta$ **then return** v
    $\alpha \leftarrow$ MAX(α, v)
  return v

Return 2

V= Max(3,2)=3
$3 \geq +\infty$ FALSE
Alfa= Max(3, 3)=3

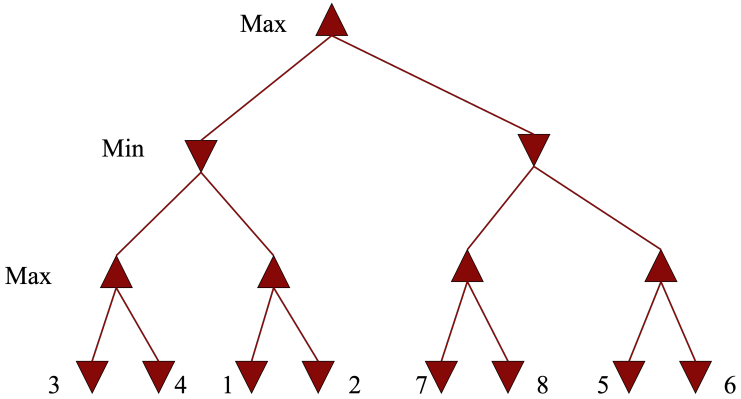
function MIN-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  if TERMINAL-TEST($state$) **then return** UTILITY($state$)
  $v \leftarrow +\infty$
  for each a **in** ACTIONS($state$) **do**
    $v \leftarrow$ MIN(v, MAX-VALUE(RESULT(s,a) , α, β))
    if $v \leq \alpha$ **then return** v
    $\beta \leftarrow$ MIN(β, v)
  return v

V= Min(5,2)=2
$2 \leq 3$ TRUE

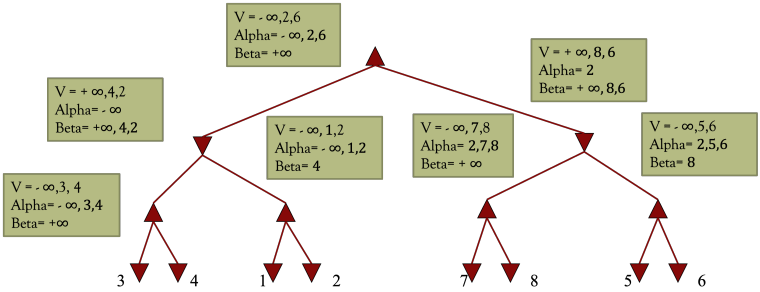
Return 2

Orden de los nodos Poda α-β

¿Cúantos nodos podemos podar?



Orden de los nodos Poda α-β

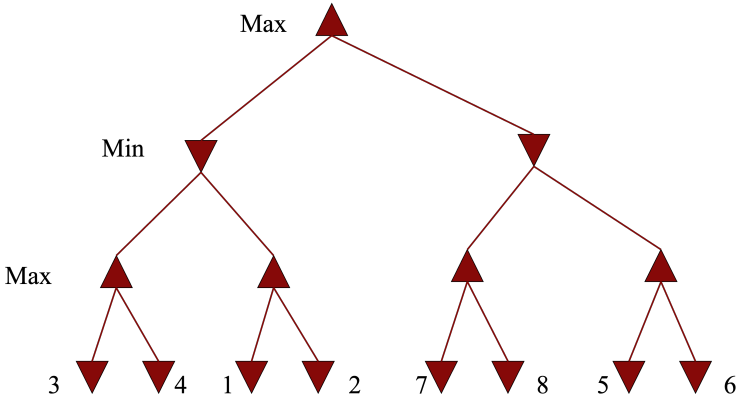


function MAX-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  if TERMINAL-TEST($state$) **then return** UTILITY($state$)
  $v \leftarrow -\infty$
  for each a **in** ACTIONS($state$) **do**
    $v \leftarrow$ MAX(v, MIN-VALUE(RESULT(s,a), α, β))
    if $v \geq \beta$ **then return** v
    $\alpha \leftarrow$ MAX(α, v)
  return v

function MIN-VALUE($state, \alpha, \beta$) **returns** *a utility value*
  if TERMINAL-TEST($state$) **then return** UTILITY($state$)
  $v \leftarrow +\infty$
  for each a **in** ACTIONS($state$) **do**
    $v \leftarrow$ MIN(v, MAX-VALUE(RESULT(s,a) , α, β))
    if $v \leq \alpha$ **then return** v
    $\beta \leftarrow$ MIN(β, v)
  return v

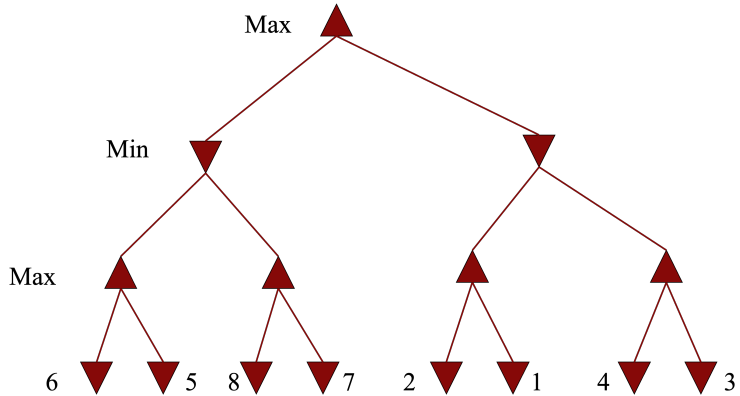
Orden de los nodos Poda α-β

No podemos podar ningún nodo. Los nodos favorables son los últimos en explorar.



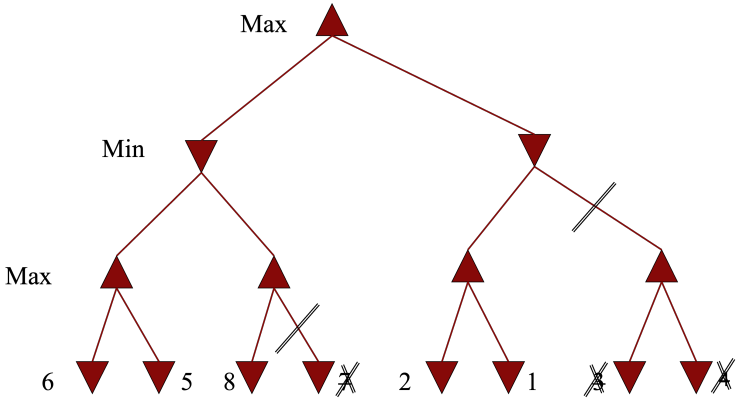
Orden de los nodos Poda α-β

¿Cúantos nodos podemos podar?



Orden de los nodos Poda α-β

Hemos podado 3 nodos.



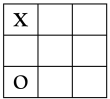
Búsqueda multiagente
Funciones de Evaluación

Funciones de Evaluación

- Minimax genera el espacio de búsqueda entero, mientras que el algoritmo alfa-beta podamos partes.
- Necesario llegar a estados terminales.
- Trabajo shannon (1950) programming a computer for playing chess, propone una función de evaluación heurística a los estados convirtiendo nodos no terminales a terminales.
- Sustituye **función utilidad** por **función evaluación heurística**, que da una estimación de utilidad de la posición y establecemos un test límite que decide cuando terminar de aplicar función.
- Su cálculo ha de ser poco costoso

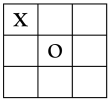
Decisiones imperfectas en juegos de dos adversarios

- Decisión imperfecta: Decisión tomada por el algoritmo sobre un horizonte que no alcanza el final del juego (se asume) y con función de evaluación estimada f = û.
- Función de evaluación, f(n)=
  1. si “n” no es terminal: (número de filas, columnas o diagonales libres para MAX) - (número de filas, columnas o diagonales libres para MIN)
  2. si gana MAX: $+\infty$
  3. si gana MIN: $-\infty$



Nº pos max = 5
Nº pos min = 5

f(n)= 5 - 5 = 0



Nº pos max = 4
Nº pos min = 5

f(n)= 4 - 5 = -1

That’s all folks!

NICS LAB